THE COMPLETE PROMPT ENGINEERING RULEBOOK

Gemini 3.8 Flash — A practical guide to accurate, efficient and consistent AI outputs

Model: Gemini 3.8 Flash **Model ID:** gemini-3.8-flash **Version:** 1.0

Purpose: achieve reliable outputs by defining the task, supplying relevant context, establishing boundaries, and specifying what success looks like.

1. Understanding Gemini 3.8 Flash

The supplied guide describes Gemini 3.8 Flash as a general-purpose model for software engineering, reasoning, agents, multimodal processing, enterprise workflows, research, analysis, multi-step problem-solving and tool-assisted tasks.

Specification	Details in supplied guide
Inputs	Text, images, audio, video and PDFs
Output	Text
Input context window	1,048,576 tokens
Maximum output	65,536 tokens
Thinking levels	LOW, MEDIUM, HIGH; default MEDIUM
Capabilities	Function calling, structured outputs, code execution, file search, Search/Maps grounding, URL context; computer

These are model specifications, not guarantees that every capability is available in every interface or API configuration.

Fundamental philosophy

Gemini benefits from direct instructions, consistent structure, explicit definitions, deliberate context placement and verbosity control, explicit multimodal references, suitable thinking levels, tool-use policies, structured output requirements, and verification.

A weak prompt names a topic. A good prompt defines a task. An excellent prompt defines the objective, context, decision rules, boundaries and success criteria.

2. Core prompting rules: clarity and structure

Rule 1 — Be precise and direct

State exactly what the model must accomplish, the subject and scope, and the intended result. Replace vague requests such as “improve this” or “research AI” with action-oriented requirements and measurable outcomes. Remove decorative language and repetition.

Rule 2 — Use one consistent prompt structure

```
# TASK
Analyse the supplied dataset.

# CONTEXT
The dataset contains monthly sales records.

# REQUIREMENTS
- Identify missing values.
- Calculate monthly revenue.
- Identify sales trends.

# OUTPUT
Provide a structured analytical report.
```

Markdown headings are usually sufficient. XML-style tags are also useful. Choose one primary structure, maintain consistent boundaries, and separate instructions from source material.

Rule 3 — Define ambiguous terms

Words such as high priority, critical, recent, significant, concise, safe, high confidence and similar can have different meanings. Define them only when their interpretation materially affects the result.

```
HIGH: prevents access to a major application feature.
MEDIUM: feature works but has substantial limitations.
LOW: cosmetic issue or minimal functional impact.
```

Rule 4 — Place critical instructions first

Put essential rules and restrictions before large context. For example: use only supplied sources; do not invent statistics or references; identify contradictions; distinguish facts from assumptions; do not modify files without authorization. For persistent application behaviour, place suitable rules in system instructions.

Rule 5 — Control verbosity explicitly

Specify not only length but the required depth, evidence, examples and interpretation. Instead of “explain everything,” request definitions, concepts, working principles, examples, applications, limitations and distinctions. When shortening, state which information must remain.

Rule 6 — Choose the appropriate thinking level

- **LOW:** simple questions, basic summaries, extraction, clear classification, routine content and latency-sensitive tasks.
- **MEDIUM:** most programming, analysis, research synthesis, business analysis, multi-step tasks and standard agents.
- **HIGH:** difficult mathematics, algorithms, complex architecture, multi-constraint optimization and deep technical investigations.

MEDIUM is the default. MINIMAL is not a supported thinking level in the supplied guide. Thinking level is an API configuration: writing “use HIGH” in a prompt does not itself change the API setting.

3. Context placement and multimodal prompting

Rule 7 — Put large context before the main question

For long documents, codebases and datasets, supply the relevant context first and place the specific task at the end. Clearly label files and sources, remove duplicates, identify the authoritative source and contradictions, and use a clear transition.

```
# CRITICAL RULES
[Essential instructions]

# CONTEXT
[Document 1]
[Document 2]
[Dataset and supporting information]

# TASK
Based on all information above, perform this analysis:
[Specific question]

# OUTPUT
[Expected format]
```

Useful transitions include “Based on the documents provided above” and “Using only the evidence supplied in the preceding sections.” A large context window does not remove the need for context management.

Rule 8 — Identify multimodal inputs explicitly

Gemini accepts text, images, audio, video and PDFs. Label every supplied asset individually and explain how it should be used. Do not refer to several attachments simply as “this.”

- **Images:** identify what the image represents, the region to inspect, details to identify and comparisons required.
- **Video:** specify timestamps, actions, events, event sequence and analysis required.
- **Audio:** specify whether transcription, speaker identification, tone analysis, summary or detailed analysis is required.
- **PDFs:** specify pages or sections, required information, tables/figures to inspect, and whether claims need passage-level support.

Example multimodal instruction

```
# INPUTS
Image A: screenshot of current application.
Image B: proposed interface.
PDF: design guidelines.
Video: mobile application recording.

# TASK
Compare the proposed interface with the current one.

# EVALUATE
Visual hierarchy, navigation, accessibility,
mobile usability and consistency with the PDF.

# EVIDENCE
Separate observable differences from recommendations.
Do not claim a violation unless the supplied PDF
contains the relevant rule.
```

Distinguish direct observations from inferences. Do not claim evidence that is not present in the supplied media or documents.

4. Advanced prompting techniques

Rule 9 — Start with zero-shot prompting

Give an instruction without examples for straightforward tasks. If the output is already accurate and consistent, extra examples are unnecessary.

Rule 10 — Use few-shot prompting when needed

Provide a small number of representative input/output examples when classification, extraction, style, formatting or decision boundaries are difficult to explain. Include borderline cases when useful; keep examples consistent and avoid excessive examples.

```
TASK: Classify messages as Billing, Access or Bug.

Input: "My card was charged twice."
Output: Billing

Input: "I cannot reset my password."
Output: Access

Input: "The application crashes when I open a project."
Output: Bug

NEW INPUT: "I paid, but my account still shows the free plan."
OUTPUT: Return one category only.
```

Rule 11 — Decompose complex tasks

Break complex work into meaningful phases when it improves reliability: requirements, architecture, database, backend, frontend, integration, testing and deployment. Use decomposition for software, research, business analysis, large datasets, mathematics and agents. Do not force stages onto simple tasks.

Rule 12 — Compare alternatives against explicit criteria

Define criteria before asking for a recommendation. For example, compare Flask and Django for a specified application using learning difficulty, development speed, performance, database integration, security, scalability, deployment and maintenance. Recommend based on the actual circumstances, not universal claims.

Rule 13 — Manage assumptions

```
If information is missing:
1. Decide whether it materially affects the result.
2. If it affects an important decision, ask a focused question.
3. Otherwise proceed with a reasonable assumption.
4. State important assumptions.
5. Never invent critical facts, identifiers, values or permissions.
```

Rule 14 — Verify important results

Define checks before completion: requirements, calculations, logical consistency, edge cases, evidence and limitations. Verification is especially important for programming, research, calculations and external actions. Do not claim an operation was tested or completed unless it actually was.

Rule 15 — Define stop conditions

Stop when mandatory requirements, deliverables and verification are complete and remaining limitations are documented. Avoid unnecessary research or unrelated modifications. For agents, request further authorization when the next action requires it.

5. Research and information accuracy

Rule 16 — Define the research objective

Avoid broad topics. Specify the subject, geography, time period, dimensions to evaluate and intended decision. Example: evaluate AI adoption in diagnostic imaging in India, focusing on applications, regulation, barriers, costs and evidence of effectiveness.

Rule 17 — Define a source hierarchy

1. Official government publications.
2. Official institutional documentation.
3. Peer-reviewed academic research.
4. University publications.
5. Official industry reports.
6. Reputable news publications.
7. Expert analysis.
8. Community discussions.

Adapt the hierarchy to the subject. Official product documentation should generally lead for technical questions; community discussions may provide supplementary qualitative experience.

Rule 18 — Define the research period

For changing information, state the exact as-of date or publication window, such as current as of November 2026, developments during 2024–2026, or historical trends from 2015–2025. Require current verification when necessary.

Rule 19 — Separate facts from interpretations

- **Verified facts:** directly supported by reliable sources.
- **Estimates:** values calculated using stated assumptions.
- **Inferences:** interpretations derived from evidence.
- **Opinions:** subjective evaluations.
- **Recommendations:** proposed actions based on available information.

Rule 20 — Require evidence and citations

For major factual claims, request supporting evidence, source identity, publication date, citation and uncertainty. Never fabricate citations, statistics or quotations. Disclose when a claim cannot be verified.

Rule 21 — Request limitations and counterarguments

Ask for supporting and contradictory evidence, alternative explanations, data and research limitations, potential biases and unresolved questions. A reliable report should not merely support one predetermined conclusion.

For source-grounded tasks, treat supplied documents as the authority when requested. Do not silently fill gaps with unsupported assumptions.

6. Programming and software development

Rule 22 — Specify the development environment

Provide the operating system, programming language and version, IDE, frameworks, libraries, database and deployment environment. Example: Windows 11, VS Code, Python, Flask, SQLite, HTML/CSS/JavaScript.

Rule 23 — Describe the existing project structure

```
project/
app.py
database.db
templates/
static/
requirements.txt

Modification rules:
- Modify only the authorized files.
- Preserve database records and user information.
- Preserve routes and existing functionality.
- Do not change unrelated files.
```

When uploaded files are involved, identify which files matter and what must remain unchanged.

Rule 24 — Specify code quality

State whether you need beginner-friendly, minimal, readable, modular, optimized or production-oriented code. Do not request advanced architecture when a simple implementation is sufficient.

Rule 25 — Define error handling

Specify behaviour for invalid inputs, missing files, duplicates, database failures, unsupported formats, empty datasets, network failures and unexpected user actions. Require meaningful error messages and safe handling.

Rule 26 — Request testing and verification

1. Syntax verification.
2. Logic verification.
3. Input validation.
4. Edge-case testing.
5. Database verification.
6. Integration testing.
7. Relevant regression tests.

Where execution tools are available, prefer actual testing over code inspection alone. Never accept claims of successful testing without evidence.

Additional coding expectations

- Understand the existing code and data flow before editing.
- Preserve existing functionality and data unless a change is explicitly authorized.
- Avoid unnecessary rewrites and dependencies.
- Use documented APIs and compatible versions.
- Report changed files, setup steps, tests performed and unverified areas.

A program is not proven correct merely because code has been generated. Verify its behaviour against the requested requirements.

7. Data analysis, mathematics and business decisions

Data analysis — Rules 27–29

Describe the dataset, file format, columns, record count if known, analysis objective and expected output. Specify the required operations: inspect data, identify missing values and duplicates, validate types, clean appropriately, calculate statistics, find patterns, visualize and conclude.

```
# IMPORTANT
Choose missing-value treatment appropriate to the data.
Do not apply mean imputation to categorical columns.
Do not invent numerical results.
```

Request numerical evidence rather than vague statements. For example, calculate the percentage increase, show the values used and identify the period.

Mathematical problem-solving — Rule 30

Specify the expected solution structure: given information, required result, relevant formula, substitution, calculations, final answer, units and verification. For complex problems, request derivations and alternative methods when useful.

Business and decision-making — Rules 31–35

Define the decision to be supported. Example: evaluate whether a ready-to-cook food business for college students in Mangaluru is feasible with ■5,000 initial capital.

- **Constraints:** capital, available time, target customers, geography, operating costs, expected revenue, resources and risk tolerance.
- **Quantitative analysis:** initial investment, fixed and variable costs, revenue, gross and net profit, break-even point, ROI and payback period.
- **Scenario analysis:** optimistic, base and pessimistic cases, with assumptions stated.
- **Recommendation:** consider feasibility, cost, risk and expected benefit—not projected revenue alone.

Clearly distinguish verified figures from estimates. Show calculations and state assumptions behind financial projections.

8. Tool usage and agentic workflows

Tool-assisted tasks need explicit policies beyond an ordinary task description.

Rule 36 — Define tool-use conditions

Specify which tools may be used, when external research is necessary, which operations are read-only, which change external state, and what information must be verified. Example: use official documentation for current API capabilities and code execution for calculations where numerical accuracy matters.

Rule 37 — Establish risk boundaries

Lower-risk activities	Higher-risk activities
Reading files; searching documentation; analysing; calculating; drafting proposals; preparing changes	Sending emails; changing passwords; deleting files; changing account settings; publishing; subm

Require explicit authorization for consequential actions according to the application's permission rules.

Rule 38 — Define recovery behaviour

1. Identify the type of failure.
2. Determine whether retrying is safe.
3. Check whether the previous operation succeeded.
4. Retry only when appropriate.
5. Record unresolved failures.
6. Never claim success when the result is unknown.

Rule 39 — Allow adaptation

Do not force an initial plan when new evidence contradicts it. Adapt the investigation while preserving the original objective.

Rule 40 — Define stop conditions

Stop when requested actions and mandatory checks are complete and limitations are documented. Ask for additional authorization if the next action requires it.

For autonomous workflows, define scope, permissions, failure recovery and completion conditions before execution.

9. Structured outputs and common mistakes

Rule 41 — Separate semantic rules from output schemas

For machine-readable responses, use Structured Outputs where supported. The prompt defines what the information means; the schema defines its format. Example: classify feedback using explicit category and severity definitions, use “unknown” when evidence is insufficient, and do not infer churn without evidence.

```
{
"type": "object",
"properties": {
"category": {"type": "string",
"enum": ["billing", "access", "bug",
"feature_request", "other", "unknown"]},
"severity": {"type": "string",
"enum": ["low", "medium", "high",
"critical", "unknown"]},
"action_required": {"type": "boolean"},
"reason": {"type": "string"}
},
"required": ["category", "severity",
"action_required", "reason"],
"additionalProperties": false
}
```

The schema is an API configuration, not a substitute for a clear prompt. Avoid duplicating complicated schemas in natural-language instructions unless needed.

Common mistakes and better approaches

Mistake	Better approach
Vague objective	Define exact outcome and scope
Exaggerated language	Use direct instructions
Mixed prompt structures	Use one consistent structure
Undefined terms	Specify decision boundaries
Critical rules buried	Place them first
Question before large context	Place final query after context
Irrelevant/duplicate files	Supply only relevant material
HIGH for every task	Calibrate thinking level
Thinking level used instead of clarity	Improve task definition
Unlabelled media	Identify each input
Tools without restrictions	Define permissions and policies
Prompt prose used for JSON validation	Use structured output schemas
Excessive examples	Use representative examples
No recovery or stop conditions	Define retry and completion rules
Prompt changes without tests	Evaluate versions on representative tasks

10. Practical Gemini 3.8 Flash prompt framework

Use all eight components for important tasks; ordinary tasks need only the smallest structure that adequately expresses the request.

1. **Critical rules:** essential instructions, source priorities and restrictions.
2. **Context:** relevant background, files, datasets, images, video and source material.
3. **Definitions:** ambiguous terms, thresholds, categories and evaluation criteria.
4. **Task:** precise objective or question.
5. **Tool and action policy:** tools, permissions, risk boundaries and recovery when relevant.
6. **Output:** format, length, structure, tone and mandatory information.
7. **Verification:** checks required before completion.
8. **Stop conditions:** when the task is finished and when clarification or authorization is needed.

Reusable master template

```
# CRITICAL RULES
[Essential instructions and restrictions.]

# CONTEXT
[Relevant background, files, datasets or source material.]

# DEFINITIONS
[Important terms, categories and thresholds.]

# TASK
[Precisely state the desired result.]

# REQUIREMENTS
1. [Requirement 1]
2. [Requirement 2]
3. [Requirement 3]

# TOOL AND ACTION POLICY
[Tools, permissions and boundaries, if applicable.]

# OUTPUT
Format: [Report/table/code/JSON/etc.]
Structure: [Required sections]
Detail: [Desired depth]
Audience: [Intended reader]

# VERIFICATION
[What must be checked.]

# STOP CONDITIONS
[Completion and clarification conditions.]
```

Short template for ordinary tasks

```
# TASK
[What must be accomplished.]

# CONTEXT
[Relevant information.]

# REQUIREMENTS
[Important instructions and boundaries.]

# OUTPUT
[Expected result and format.]
```

Do not include every section automatically. Use the smallest prompt structure that preserves all information needed for a reliable result.

11. API configuration: important technical distinctions

This section applies to developers integrating Gemini 3.8 Flash through an API. Prompt instructions and API configuration are different things. The following details are stated in the supplied rulebook and should be checked against current official documentation before implementation.

Thinking level

The supplied documentation lists LOW, MEDIUM and HIGH; MEDIUM is the default and MINIMAL is unsupported. Configure thinking level through the supported API configuration, not merely through natural-language prompt text.

Sampling parameters

The supplied developer documentation states that these legacy parameters are ignored by the backend for this model:

- temperature
- top_k
- top_p

Unsupported parameters

The supplied documentation identifies the following as unsupported and says passing them causes API errors:

- frequency_penalty
- presence_penalty
- candidate_count

Function calling

Function responses must correctly match the preceding function call. Keep the function-call identifier, function name and execution sequence consistent.

Multimodal inputs

Place media assets in the appropriate request payload. Describing an image or video in plain text is not equivalent to supplying the actual media.

Conversation history

- History payloads cannot end with a model-role turn.
- Prefilled model turns are unsupported.
- Empty conversation turns may be dropped or cause validation errors.

General API recommendation

Use the current official SDK and documentation for exact request syntax. Do not copy legacy API parameters into a new model integration without checking compatibility.

API parameter behaviour is version-sensitive. Treat the supplied parameter list as source-document guidance, not a substitute for checking the current API reference.

12. Prompt evaluation and optimization

Evaluate prompts by actual performance, not by how sophisticated they look.

Establish a baseline

Start with a simple prompt, test it against representative tasks and record the results.

Measure performance

1. Task completion and instruction following.
2. Factual accuracy and evidence grounding.
3. Output consistency and formatting reliability.
4. Tool-use accuracy and error recovery.
5. Response latency, token consumption and overall cost.

Compare thinking levels where useful

For tasks where reasoning effort may matter, compare the same prompt with LOW, MEDIUM and HIGH while keeping other variables unchanged. Determine whether additional reasoning effort actually improves results.

Test difficult cases

- Incomplete information and conflicting evidence.
- Invalid inputs, unusual cases and ambiguous requests.
- Large datasets and long context.
- Tool failures and uncertain operation outcomes.

Maintain prompt versions

1. Save the original prompt.
2. Make one meaningful change.
3. Test the revised version on the same representative cases.
4. Compare results and retain the better version.
5. Repeat evaluation when the model or configuration changes.

A shorter prompt is not automatically better; a longer prompt is not automatically better either. The objective is reliable performance.

Measure changes against a baseline. Do not assume that a prompt improvement works until representative testing supports it.

13. Gemini 3.8 Flash TTS — separate prompting rules

This section applies to the separate speech-generation models gemini-3.8-flash-tts and gemini-3.8-flash-lite-tts, not the general-purpose gemini-3.8-flash model.

Separate the three components

Component	Purpose
Voice	Who is speaking; stable identity
speech_metadata.style	How the current turn is delivered
text	What the speaker actually says

Transcript and delivery

Treat the text field as the verbatim transcript. Do not put long acting directions or “say this in an excited voice” inside the spoken text. Use speech_metadata.style for sustained delivery such as friendly, slow and clear, urgent whisper, sarcastic, or formal presenter. Style may control emotion, pace, pitch, prosody and inflection; keep it short and relevant.

Inline vocal events and pauses

Use angle-bracket tags for momentary vocal events and pauses, such as , , , , and . These are for human vocalisations and pauses, not arbitrary environmental sound effects. Use natural punctuation first and avoid excessive tags.

Emotion, emphasis and natural speech

Use style for sustained emotion and pitch. For major emotional changes, divide dialogue into separate turns with their own style. Capitalization and punctuation can emphasize selected words; avoid all-caps paragraphs. Write natural spoken language and use disfluencies only where they improve conversational realism.

Voice identity and consistency

Use Voice Design for stable characteristics: timbre, pitch, accent, baseline cadence, diction and resonance. Reuse the resulting voice ID. Do not repeatedly put long identity descriptions in style or use style to change stable traits such as age, gender or permanent accent. Rely on the configured voice reference for identity consistency.

Two-speaker backchannels

For supported multi-speaker dialogue, pipe-delimited short listener reactions may be used, for example: “So the launch is Thursday |oh really| Are we actually ready?” Short reactions include |mhm|, |oh really?|, |wait| and |yeah|. The supplied guide notes this particularly for Gemini 3.8 Flash TTS.

Model selection and workflow

Flash TTS: studio-grade acting, nuanced emotion, regional dialects, multi-speaker scenes and long-form narrative stability.
Flash-Lite TTS: high-volume generation, real-time agents, customer support and read-aloud applications. The supplied guide says both use the same basic API structure and prompting format.

1. Select or design the voice.
2. Prepare a natural transcript and test with an empty style.
3. Listen to the baseline; add short style instructions only if needed.
4. Add inline vocal tags appropriately; split major emotional changes into turns.
5. Test the final output.

TTS principle: separate the script from the performance direction.

14. Final checklist, 25 golden rules and philosophy

Final prompt checklist

- **Objective/context:** objective, scope and relevant background are clear; irrelevant context is removed.
- **Structure:** instructions are organized consistently; critical rules are first; important terms are defined.
- **Thinking:** LOW, MEDIUM or HIGH is justified by task complexity; unnecessary complexity is avoided.
- **Long context:** assets are labelled; large context precedes the final query; a clear transition and source priorities are provided.
- **Multimodal:** each image, video, audio file or PDF is identified and its intended use explained; observations are separated from inferences.
- **Output:** format, detail, mandatory sections and audience are specified.
- **Reliability:** evidence, assumptions, verification and limitations are addressed.
- **Tools/agents:** tool conditions, permissions, risk boundaries, failure recovery and stop conditions are defined.
- **Evaluation:** prompt is tested on representative cases; edge cases and thinking levels are evaluated where useful; versions are maintained.

The 25 golden rules

1. Be direct and precise; define the exact outcome.
2. Supply relevant context and use one consistent structure.
3. Place critical instructions first.
4. Define ambiguous terms and parameters.
5. Specify output format and verbosity.
6. Select an appropriate supported thinking level.
7. Place large context before the final query.
8. Use a clear transition between context and task.
9. Avoid irrelevant or duplicated context.
10. Label multimodal inputs explicitly.
11. Start with zero-shot prompting.
12. Use few-shot examples when they clarify behaviour.
13. Decompose complex tasks when useful.
14. Define source priorities for research.
15. Distinguish facts, estimates, assumptions and inferences.
16. Specify the technical environment for programming.
17. Define tool permissions and risk boundaries.
18. Specify recovery behaviour for agentic tasks.
19. Separate semantic rules from output schemas.
20. Define verification and completion criteria.
21. Test prompts using representative tasks.
22. Do not use HIGH thinking for every task.
23. Do not fill the context window unnecessarily.
24. Do not add examples or prescribe obvious steps without reason.
25. Use the smallest prompt and configuration that reliably accomplish the task.

FINAL PHILOSOPHY: Clearly define the task, organize relevant context, establish boundaries and specify the expected result. Configure thinking and tools appropriately. Test the result, identify weaknesses and refine the prompt. Efficient prompting is not about maximum length; it is about reliable performance with no unnecessary instructions.